Funzioni e funzione main

Marco Cerbella

Lezione 16 – 3 novembre 2026

1 Funzioni

1.1 Introduzione e main

Tutte le istruzioni di un programma C sono contenute in funzioni. Il C è un linguaggio procedurale: ogni funzione svolge un certo compito. Un nome speciale di funzione è `main()`: la funzione con questo nome è la prima a essere eseguita quando il programma parte. Il primo comando eseguito nel programma è il primo comando della `main` (dall'alto verso il basso). Tutte le altre funzioni sono sottoprogrammi della `main()` e possono avere i nomi che si vuole.

Ogni funzione è definita esattamente una volta. Un programma può dichiarare e chiamare una funzione tante volte quante ne servono.

1.2 Definizione di funzione

La definizione di una funzione consiste in un'*intestazione* (o dichiaratore) e in un *blocco* della funzione. L'intestazione specifica il nome della funzione, il tipo del valore restituito e i tipi e i nomi dei parametri, se ce ne sono. Le istruzioni nel blocco specificano che cosa fa la funzione.

```
tipo nome_funzione (tipo nome_parametro1, tipo nome_parametro2, ...) {
    // istruzioni;
}
```

Osservazione. Scrivere sempre il tipo del valore restituito: se non è specificato è `int`.

1.3 Dichiarazione di funzione

Dichiarando una funzione prima di usarla si informa il compilatore del suo tipo: la dichiarazione descrive cioè l'*interfaccia* della funzione. Una dichiarazione definisce

- il tipo della funzione;
- il numero e i tipi dei suoi parametri;
- il tipo del valore restituito.

Gli identificatori dei parametri in una dichiarazione di funzione sono facoltativi. Se si scrivono i nomi, il loro scope termina con il prototipo stesso. Le due forme sono entrambe valide:

```
tipo nome_funzione (tipo nome_parametro1, tipo nome_parametro2, ...);
tipo nome_funzione (tipo, tipo, ...);
```

1.4 Scope degli identificatori

- *Block scope*: gli identificatori dichiarati in un blocco hanno block scope. Si può usare un tale identificatore solo dalla sua dichiarazione fino alla fine del blocco più piccolo che contiene la dichiarazione. Il blocco più piccolo che la contiene è spesso, ma non necessariamente, il corpo di una definizione di funzione. In C11 le dichiarazioni non devono essere necessariamente poste prima di tutte le istruzioni nel blocco di una funzione. Anche i nomi dei parametri nell'intestazione di una definizione di funzione hanno block scope, e sono validi nel blocco corrispondente della funzione.
- *Function prototype scope*: i nomi dei parametri in un prototipo di funzione hanno function prototype scope. Poiché questi nomi non hanno significato al di fuori del prototipo stesso, hanno valore solo di commento e si possono anche omettere.

Esempio.

```
1 int p;
2 int fun1(int q);
3 int fun2(int r)
4 {
5     int s = r + 3;
6     while (r != s)
7         r--;
8 }
```

`p`, `fun1` e `fun2` sono visibili in tutto il file (da dove sono dichiarati); `q` ha scope il solo prototipo; `r` e `s` hanno scope il blocco di `fun2`.

1.5 Parametri di una funzione

I parametri di una funzione sono normali variabili locali. Il programma li crea e li inizializza con i valori dei corrispondenti argomenti quando avviene una chiamata di funzione. Il loro scope è il blocco della funzione (block scope).

```
1 long double factorial(unsigned int n)
2 {
3     long double f = 1;
4     while (n > 1)
5         f *= n--;
6     return f;
7 }
```

1.6 Dichiarazione e definizione

Una definizione di funzione è anche una dichiarazione di funzione. Le due versioni seguenti sono equivalenti: nella prima `fun2` è dichiarata prima della `main` e definita dopo; nella seconda è definita prima della `main`, e la definizione fa anche da dichiarazione.

```
1 int fun2(int);
2 int main() {
3     int a = 5;
```

```
    a = fun2(a);
}
int fun2(int r) {
    int s = r + 5;
    while (r != s)
        r--;
    return r;
}
```

```
int fun2(int r) {
    int s = r + 5;
    while (r != s)
        r--;
    return r;
}
int main() {
    int a = 5;
    a = fun2(a);
}
```

In sintesi:

- prima di chiamare (usare) una funzione bisogna dichiararla;
- si possono dichiarare tutte le funzioni all'inizio del file e definirle in un ordine qualunque più sotto nello stesso file; oppure si possono evitare le dichiarazioni in testa, ma allora vanno definite *prima* di essere chiamate;
- una funzione deve anche essere definita, altrimenti non si sa che cosa fare quando viene chiamata;
- nello stesso file non si possono avere due definizioni di funzioni con la stessa intestazione e lo stesso identificatore (nome).

Esempio (Dichiarazioni e ordine delle definizioni)**.** Questa versione è corretta: `fun1` e `fun2` sono dichiarate in testa.

```
int fun1(int);
int fun2(int);
int main() {
    int a = 5;
    a = fun1(a);
}
int fun1(int q) {
    return (fun2(q) + 1);
}
int fun2(int r) {
    int s = r + 5;
    while (r != s)
        r--;
    return r;
}
```

Questa invece è *sbagliata*: `fun1` chiama `fun2`, che non è stata dichiarata prima di quel punto (e non è definita prima).

```c
int fun1(int);
int main() {
    int a = 5;
    a = fun1(a);
}
int fun1(int q) {
    return (fun2(q) + 1);   // errore: fun2 non dichiarata
}
int fun2(int r) {
    int s = r + 5;
    while (r != s)
        r--;
    return r;
}
```

Si può correggere dichiarando `fun2` in testa, oppure definendola prima di `fun1`.

1.7 return

L'istruzione `return` termina l'esecuzione della funzione corrente e salta indietro al punto in cui la funzione era stata chiamata:

```
return [espressione];
```

L'espressione viene valutata e il risultato viene restituito al chiamante come valore della chiamata di funzione. Questo valore restituito viene convertito al tipo restituito dalla funzione, se necessario. Una funzione può contenere un numero qualunque di istruzioni `return`; il primo `return` eseguito fa uscire dalla funzione. `return` è un'istruzione di salto incondizionato (come `goto`).

Esempio.

```c
// Restituisce il minore di due argomenti interi.
int min(int a, int b)
{
    if (a < b)
        return a;
    else
        return b;
}
int main()
{
    int a = 5;
    int b = 6;
    int c = min(a, b);
    printf("%d", c);
}
```

```
5
```

Un `return` senza espressione può essere usato solo in una funzione di tipo `void`; anzi, queste funzioni non hanno nemmeno bisogno di un'istruzione `return`. Se in una funzione non si incontra nessun `return`, il flusso del programma torna al chiamante quando si raggiunge la fine del blocco della funzione.

2 La funzione main

Si può definire la funzione `main()` in una delle due forme seguenti:

```
int main(void) { /* ... */ }                    // nessun parametro, restituisce
    int
int main(int argc, char *argv[]) { /* ... */ } // due parametri di tipo int e
    char **, restituisce int
```

2.1 Parametri della main

I parametri `argc` e `argv` (ai quali si possono dare altri nomi) rappresentano gli *argomenti sulla riga di comando* del programma. Funzionano così:

- `argc` (da *argument count*) è 0 oppure il numero dei token stringa nella riga di comando che ha avviato il programma. Il nome del programma stesso è incluso in questo conteggio;
- `argv` (da *argument vector*) è un array di puntatori a `char` che puntano ai singoli token stringa ricevuti sulla riga di comando. Il numero di elementi di questo array è uno in più del valore di `argc`: l'ultimo elemento, `argv[argc]`, è sempre un puntatore nullo;
- la prima stringa, `argv[0]`, contiene il nome con cui è stato invocato il programma. Se l'ambiente di esecuzione non fornisce il nome del programma, la stringa è vuota;
- se `argc` è maggiore di 1, le stringhe da `argv[1]` a `argv[argc - 1]` contengono gli argomenti sulla riga di comando del programma.

Esempio (Un esempio classico)**.**

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc == 0)
        puts("No command line available.");
    else {
        printf("The program now running: %s\n", argv[0]);  // nome del programma
        if (argc == 1)
            puts("No arguments received on the command line.");
        else {
            puts("The command line arguments:");
            for (int i = 1; i < argc; ++i)    // un argomento per riga
                puts(argv[i]);
        }
    }
}
```

Se su un sistema Unix si lancia il programma con la riga di comando

```
$ ./args one two "and three"
```

l'output è

```
The program now running: ./args
The command line arguments:
one
two
and three
```

2.2 Valore restituito dalla main

Il valore restituito dalla `main` viene passato al processo padre che l'ha eseguita (per esempio la shell). Il valore restituito dalla `main` dovrebbe indicare come è terminato il programma: l'uscita normale è generalmente rappresentata dal valore 0; una terminazione anomala è di solito segnalata da un valore diverso da zero.

3 Come vengono eseguite le funzioni

3.1 Uscire da una funzione

Se il programma raggiunge un'istruzione `return` o la parentesi graffa di chiusura `}` del blocco della funzione, l'esecuzione della funzione termina e il programma torna alla funzione chiamante. Se il programma «esce dal fondo» della funzione raggiungendo la graffa di chiusura, il valore restituito al chiamante è *indefinito*. Per questo motivo bisogna usare un `return` per terminare qualunque funzione che non abbia tipo `void`. Il valore dell'espressione di `return` viene restituito alla funzione chiamante.

3.2 Chiamare una funzione

L'istruzione che esegue una funzione, la *chiamata di funzione*, consiste nel nome della funzione e nell'operatore `()`. Per esempio, l'istruzione seguente chiama la funzione `maximum()` per calcolare il massimo di due numeri:

```
1 maximum(a, b);
```

Il programma prima alloca lo spazio di memoria per i parametri, poi copia i valori degli argomenti nelle locazioni corrispondenti. Quindi salta all'inizio della funzione, e l'esecuzione della funzione comincia con la prima definizione di variabile o istruzione del blocco.

Esempio.

```
1 #include <stdio.h>
2 int sum(int p, int q) {
3     int r = 0;
4     r = p + q;
5     return r;
6 }
```

```c
int main(void) {
    int i = 3, j = 5, s = 0;
    s = sum(i, j);
    printf("%d\n", s);
    return 0;
}
```

```
8
```

Al momento della chiamata, `p` e `q` ricevono copie dei valori di `i` (3) e `j` (5): `i` e `j` sono passati *per valore*. `r` vale prima 0 e poi 8, che viene restituito e assegnato a `s`.

3.3 Parametri formali e attuali

Quando una funzione viene chiamata, i valori (le espressioni) passati nella chiamata si chiamano *argomenti* o *parametri attuali* (i due termini hanno lo stesso significato). Al momento della chiamata, ogni parametro attuale viene assegnato al corrispondente *parametro formale* nella definizione della funzione. Nell'esempio precedente `p` e `q` sono i parametri formali; i valori di `i` e `j` sono i parametri attuali.

3.4 Allocazione delle variabili

L'*allocazione di memoria* significa che una parte della memoria viene riservata, per esempio per una variabile. Una *variabile automatica* è una variabile locale che viene allocata e deallocata automaticamente quando il flusso del programma entra nel suo scope e ne esce. Per *deallocazione* si intende che quella parte di memoria non è più riservata per quella variabile. Le variabili `p` e `q` vengono allocate quando la funzione è chiamata e deallocate quando se ne esce.

4 Overloading di funzioni

L'*overloading* è una caratteristica di un linguaggio di programmazione che permette di avere più funzioni con lo stesso nome ma con firme diverse (tipo dei parametri e tipo restituito). È presente nella maggior parte dei linguaggi orientati agli oggetti, come C++ e Java. **Il C non supporta questa caratteristica.** Per fare l'overloading di una funzione in C++ la si può definire più volte, ciascuna identificata in modo univoco dal tipo degli argomenti che accetta; il tipo restituito non viene considerato.

Esempio.

```c
#include <stdio.h>
int min(int x, int y) {
    return (x < y ? x : y);
}
float min(float x, float y) {
    return (x < y ? x : y);
}
int main() {
    int a = min(4, 1);
    float b = min(4.56F, 1.23F);
}
```

Compilando con `gcc` si ottiene l'errore `conflicting types for 'min'`; compilando lo stesso file come C++ (`g++`) funziona.

5 Riferimenti

Sul libro: sezioni 5.1–5.6, 15.3, 7.4.